ThermalGuard: Live Demo Reference & Presentation Guide

Workload-Aware NVMe Throttling via cgroups v2 · Rate-of-Rise (dT/dt) Predictive Governor · Heuristic Quarantine

THE 30-SECOND ELEVATOR PITCH (READ THIS TO OPEN YOUR DEMO):

"Modern NVMe SSDs suffer from a major disconnect: when heavy background writes overheat the drive, internal firmware throttles clock speeds *indiscriminately*, causing latency-critical reads (databases, games) to hit a tail-latency cliff. Meanwhile, OS cgroups schedulers are *thermally blind*.
ThermalGuard bridges this gap: it monitors thermal accumulation, autonomously classifies processes into priority cgroups, and uses a novel **Rate-of-Rise (dT/dt) Predictive Governor** to proactively throttle only background writes. This achieves **zero thermal overshoot**, eliminates sawtooth oscillations, and guarantees strict latency SLAs for critical apps."

1. Core Architecture & Important Files (Where the Logic Lives)

File	Role	Key Algorithms & Core Implementation Logic
policy.py	Governor Decision Engine	**The Novelty Core:** Tracks thermal derivative ΔT/Δt ($^\circ ext{C/s}$). Projects temperature: T_proj = T + max(0, dT_dt) · τ_lookahead (5s horizon). Pre-emptively clamps background bandwidth before 70°C is hit. Eliminates thermal overshoot & sawtooth hunting.
classifier.py	2-Layer Process Classifier	**Autonomous Partitioning:** • *Layer 1 (Rules):* Prefix regex on /proc/<pid>/comm and cmdline. • *Layer 2 (Heuristic):* 3s window on /proc/<pid>/io. If write ratio >70%, quarantines process to tg/background. • *Anti-Flapping:* Bursts <5MB left unassigned in root (no false positives).
thermal.py	Heat Model / Sensor	Differential thermal model: dT/dt = a·wMBps - b·(T - T_amb). Equilibrium at 85°C full speed. Reads physical NVMe hwmon input if available on real hardware.
monitor.py	Kernel Telemetry	Reads Linux cgroups v2 kernel interfaces at 1 Hz: reads actual read/write byte rates from io.stat and I/O stall pressure from io.pressure (PSI).
actuator.py	Cgroups v2 Enforcer	Applies bandwidth limits by writing <device> wbps=<limit> to /sys/fs/cgroup/tg/background/io.max. Always resets to max on exit/signals.
dashboard.py	Web Server & UI	Standard library HTTP server on port 8000. Coordinates experiment subprocesses, streams live logs, and drives the dynamic SVG/Canvas pipeline visualization.

2. Dashboard Controls & Specialized Lab Demonstration Pages

Control / Lab	Mode / Type	What It Executes & What to Explain
■ **ThermalGuard (Auto-Partition)**	**Flagship Novelty Run (Two-Phase Execution)**	**Phase 1:** Spawns 4 realistic workloads into root cgroup (game loader, bulk sync, MySQL DB, tar archive). The animated pipeline scans, classifies, and divides them into tg/critical (2) and tg/background (2). **Phase 2:** Launches Rate-of-Rise Predictive Governor. Caps background writes smoothly at ~68°C. **Zero overshoot!**
■ **Baseline (No Control)**	**Unmanaged System (Raw Hardware Run)**	No throttling (wbps=max). Drive temperature rockets past 85–95°C. Tail latency (p99) on critical reads jumps by 40–80%, demonstrating the hardware thermal cliff.
■ **Stop**	**Emergency Halter**	Terminates active workload processes, halts background classifier thread, and resets io.max=max.
■■ **Rogue Binary Demo ■ (Dedicated Lab)**	**Standalone Page /demo-rogue**	Tests unprofiled binary /tmp/photo_sync doing 2 GB/s writes alongside a short 15MB burst. • *Layer 1 Rules:* Misses photo_sync. • *Layer 2 Heuristic:* Detects sustained >70% write ratio & **quarantines** it to tg/background. • *Anti-Flapping:* Transient 15MB burst is NOT quarantined (avoids false positives).
■ **Reactive Lab ■ (Dedicated Lab)**	**Standalone Page /demo-reactive**	Side-by-side comparison of Legacy Threshold Reactive Governor vs Predictive Governor. **Physics Flaw:** Legacy 60/70°C threshold governor reacts too late; thermal inertia drives temp over 70°C (overshoot of +2 to +4°C) with sawtooth oscillation, whereas Predictive clamps smoothly.
■ **5 Test Cases ■ (Evaluation Lab)**	**Interactive Lab & Automated Suite**	Executes all 5 test cases contrasting Normal vs ThermalGuard scenarios (Read SLA, Cgroups Division, Rogue Quarantine, Anti-Flapping Burst Guard, Momentum Control). Run: python3 tests/run_all_test_cases.py

3. Academic Evaluation Matrix (The Scientific Defense)

This is the core evaluation table that defends the project against reviewers and proves its Pareto optimality:

Metric	Baseline (None)	Reactive Gov	TG Predictive (Ours)	Rogue Quarantined
Peak Temperature	85–95°C (Severe)	72–74°C (Warm)	**68–69°C (Safe)**	68–70°C (Safe)
Thermal Overshoot?	Yes (+15 to 25°C)	Yes (+2 to 4°C)	**Zero (0.0°C)**	Zero (0.0°C)
Critical p99 Latency	55–75 µs (Degraded)	36 µs (Protected)	**34 µs (Protected)**	35 µs (Protected)
Throughput Stability	Runaway until throttle	Sawtooth Oscillations	**Smooth Proactive Clamping**	Smooth Proactive Clamping
Workload SLA Status	SLA Violated (Cliff)	Oscillating Latencies	**Optimal Pareto Front**	**Rogue Isolated**

4. Step-by-Step Live Demo Presentation Script

Execute these 5 steps in order during your evaluation demo:

Step	Action on Dashboard	What to Say & Point Out to the Evaluator
STEP 1	Start daemon & open UI: sudo python3 dashboard.py Open http://localhost:8000	"Here is the ThermalGuard primary control dashboard. It provides real-time observability into simulated NVMe temperature, cgroup read/write bandwidth, and our automated kernel process classification pipeline."
STEP 2 (The Problem)	Click ■ **Baseline (No Control)** (Run for ~30–45 seconds)	**What to point out:** "Notice the red temperature line shooting up towards 85–90°C. Because background writes are uncontrolled, the drive accumulates severe thermal debt and critical read latency spikes. This is the hardware thermal cliff."
STEP 3 (The Flagship)	Click ■ **ThermalGuard (Auto-Partition)**	**What to point out:** **1. Phase 1 (Animated Pipeline):** "Watch 4 processes launch into root. Our classifier inspects their command line and I/O profile, cleanly dividing them: games and DB to critical (count: 2); sync and log tar to background (count: 2)." **2. Phase 2 (Predictive Governor):** "Notice status switches to 'ThermalGuard Active'. Our rate-of-rise governor calculates dT/dt, pre-emptively clamps background writes, and levels off at 68°C with **zero thermal overshoot** while game reads stay fast."
STEP 4 (Heuristic Quarantine)	Click ■■ **Rogue Binary Demo** ■ (Opens /demo-rogue)	**What to point out:** "We created a dedicated lab for zero-day rogue binary evaluation. We launch /tmp/photo_sync. Layer 1 rules miss it. Layer 2 heuristic inspects /proc/<pid>/io, detects sustained heavy writes, and dynamically **quarantines** it to tg/background. Notice the transient burst was NOT quarantined, demonstrating our anti-flapping guard."
STEP 5 (Thermal Momentum Lab)	Click ■ **Reactive Lab** ■ (Opens /demo-reactive)	**What to point out:** "In this dedicated physics lab, we demonstrate why traditional reactive governors fail. Even though it throttles at 70°C, thermal momentum causes temperature to overshoot to 73°C and enter a chaotic sawtooth oscillation. ThermalGuard's dT/dt lookahead completely prevents this overshoot."